Nagarjuna College of Information Technology

Introduction to Cloud Computing

CSC 467

Lab Report

Submitted by:

Kailash Badu

Roll number:09

B.Sc.CSIT

Eight semester

Symbol:28986/078

Submitted to:

Department of TU

Table of Content

S.N	Title	Date	Signature
1	Demonstrate the Installation of Tomcat Server in Java with proper configurations along with the username and password.		
2	Demonstrate the Testing of the installed Tomcat Server with authentication and display the server status.		
3	Creating a Virtual Machine on Azure and uploading a file to that Virtual Machine		
4	Hosting a Static Site using Azure Blob Storage		
5	To get familiar with Azure Services.		
6	Write a program to implement multithreading in Java.		
7	Write a program to implement task programming in .NET.		
8	Write a program to implement Hadoop / MapReduce in .NET.		

9	Write a program to implement simple Task Programming in Java and execute it in a Virtual Machine (VM).		
10	Run a Simple C Program in Virtual Machine using Type2 Virtualization.		
11	Simulate a cloud scenario using cloud Sim and a scheduling algorithm that is not present in cloud Sim.		
12	Create a SOAP based web service “calculator service” in java and demonstrate the result.		

LAB 1

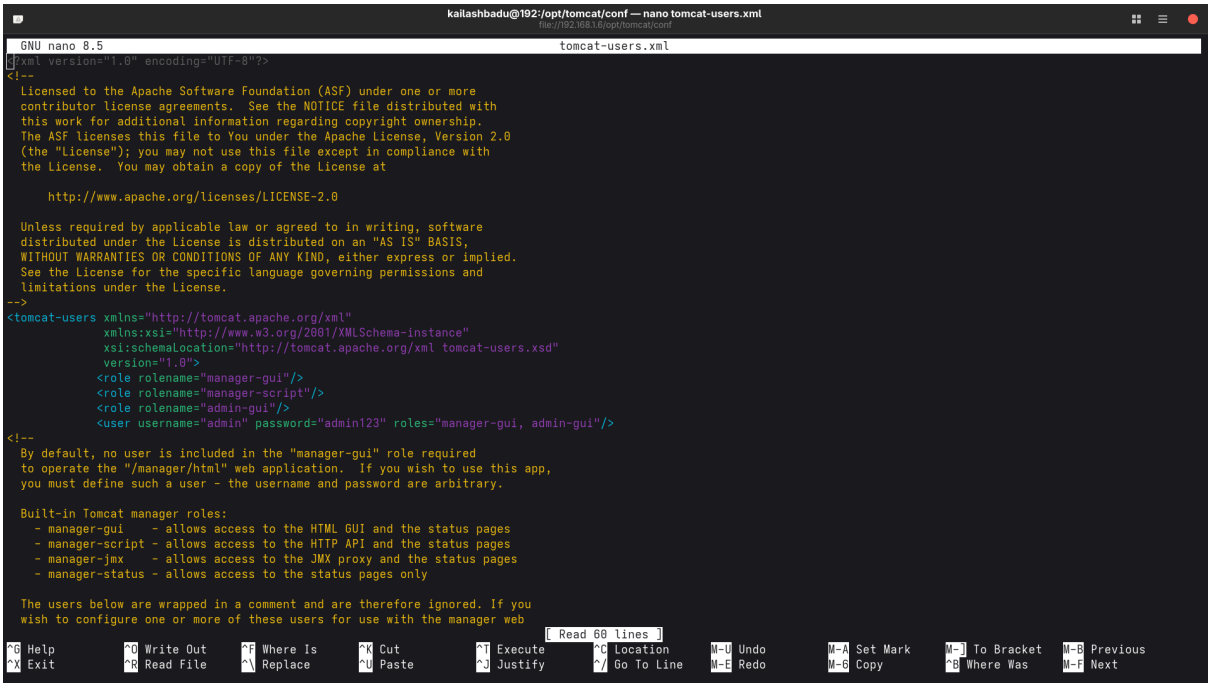
i. Demonstrate the Installation of Tomcat Server in Java with proper configurations along with the username and password.

Theory

Apache Tomcat is a web server and application server that is used to run Java-based web applications. It takes requests from a browser, processes them using Java technologies like Servlets and JSP, and sends back web pages as responses. In simple terms, Tomcat acts as a middle layer between the user's browser and Java web applications, allowing dynamic websites to run on a server.

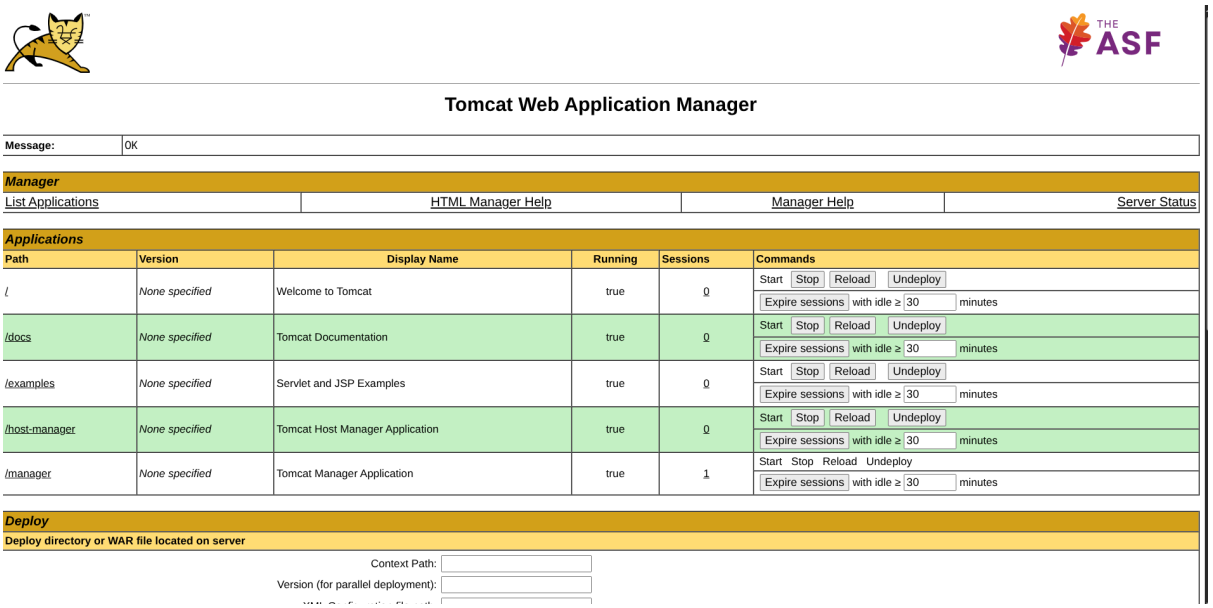
Steps - To Install Tomcat Server

1. Download and install JDK (8 or higher)
2. Download the latest version of Apache Tomcat from the official website.
3. Start the Tomcat Server. Go to the bin folder inside Tomcat directory and run: startup.bat.
4. To configure Username and Password, open conf/tomcat-users.xml and add inside <tomcat-users>.



5. Restart the Tomcat Server.

6. Access the Tomcat Manager through http://localhost:8080/manager/html.



Tomcat Web Application Manager

Message:	OK

Manager

List Applications	HTML Manager Help	Manager Help	Server Status

Applications

Path	Version	Display Name	Running	Sessions	Commands
/	*None specified*	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	*None specified*	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	*None specified*	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	*None specified*	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	*None specified*	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

Deploy

Deploy directory or WAR file located on server

Context Path:

Version (for parallel deployment):

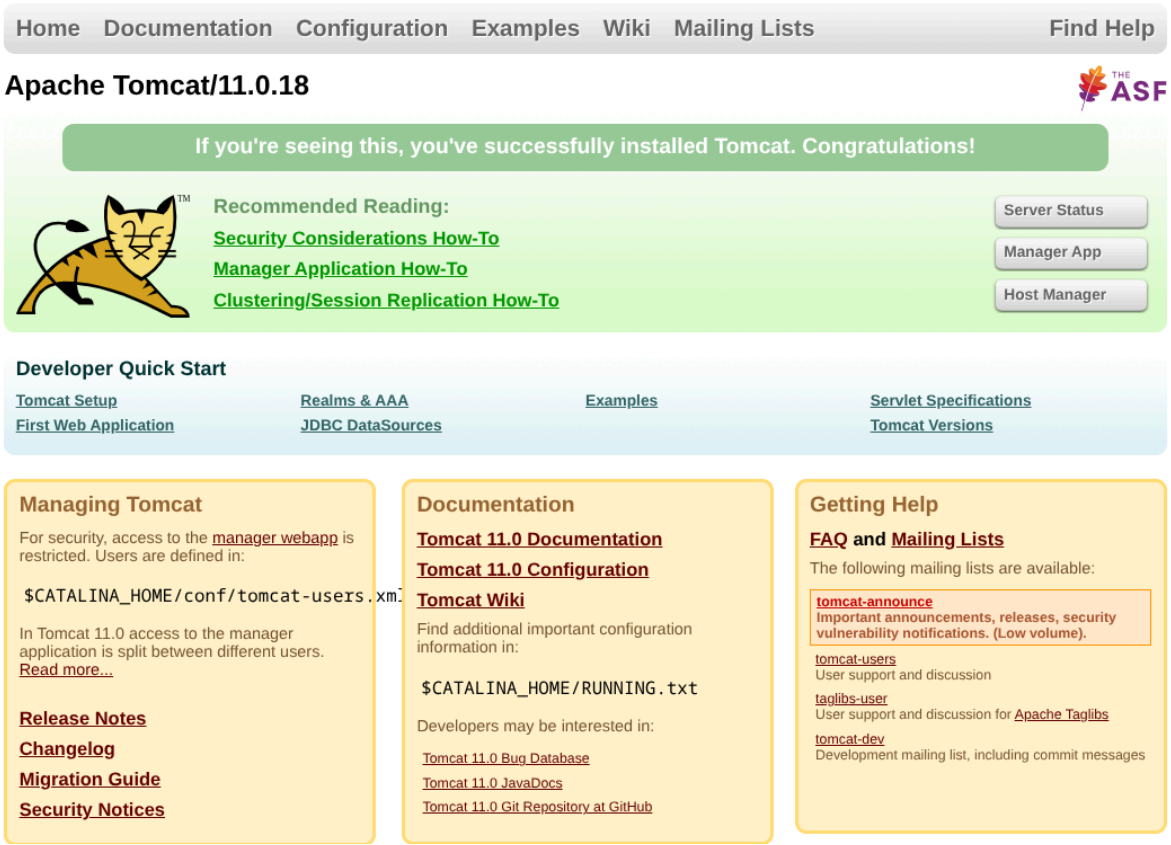
Conclusion

This demonstration showed how to install Apache Tomcat, configure environment variables, start the server, and set authentication credentials for secure access. The server was successfully configured with a username and password, enabling controlled administrative access through the Tomcat Manager interface.

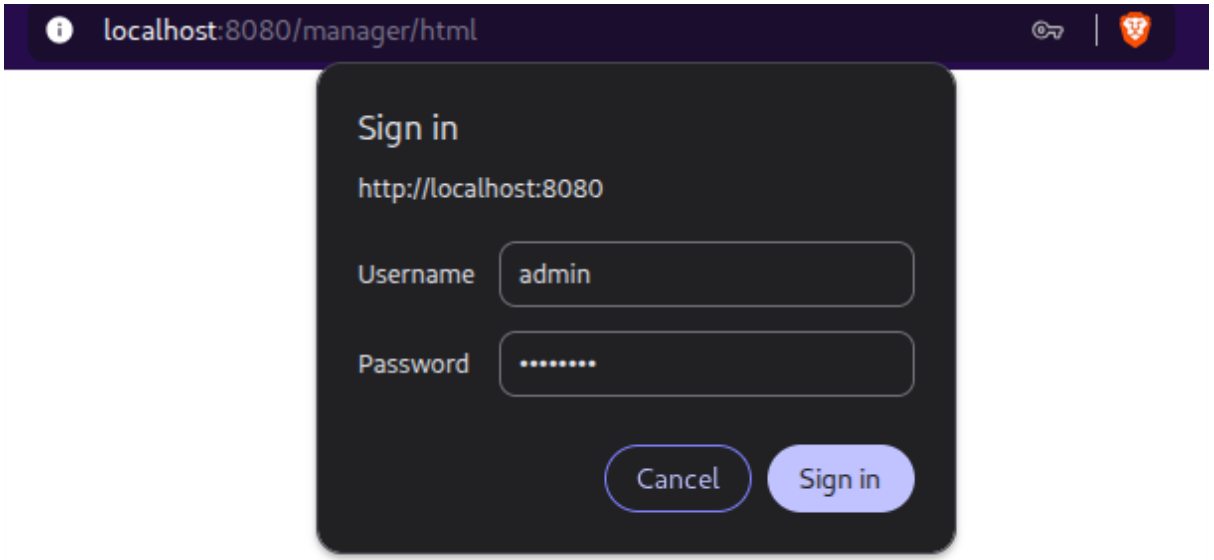
ii. Demonstrate the Testing of the installed Tomcat Server with authentication and display the server status.

Steps - Testing Apache Tomcat Server

1. Start the Tomcat Server. Go to the bin folder inside Tomcat directory and run: startup.bat.
2. Open the Tomcat Web Interface through http://localhost:8080.



3. Access the Tomcat Manager Page through http://localhost:8080/manager/html and authenticate the user with the configured credentials from tomcat-users.xml.



4. After authentication, the Tomcat Manager Dashboard will appear, confirming that authentication is working correctly.

Message:	OK

Manager

List Applications	HTML Manager Help	Manager Help	Server Status

Applications

Path	Version	Display Name	Running	Sessions	Commands
/	*None specified*	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	*None specified*	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	*None specified*	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	*None specified*	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	*None specified*	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

5. Open http://localhost:8080/manager/status to access Server Status Page. This page displays detailed server status such as JVM memory usage, request processing, and thread information.

Server Status

Manager

List Applications	HTML Manager Help	Manager Help	Complete Server Status

Server Information

Tomcat Version	JVM Version	JVM Vendor	OS Name	OS Version	OS Architecture	Hostname	IP Address
Apache Tomcat/11.0.18	25.0.2+10	Red Hat, Inc.	Linux	6.18.16-200.fc43.x86_64	amd64	192.168.1.6	192.168.1.6

JVM

Free Memory: 80.66 MiB Total Memory: 122.00 MiB Max Memory: 1898.00 MiB

Memory Pool	Type	Initial	Total	Maximum	Used
G1 Eden Space	Heap memory	47.00 MiB	49.00 MiB	-0.00 MiB	31.00 MiB
G1 Old Gen	Heap memory	73.00 MiB	67.00 MiB	1898.00 MiB	2.67 MiB (0%)
G1 Survivor Space	Heap memory	0.00 MiB	6.00 MiB	-0.00 MiB	6.00 MiB
CodeHeap 'non-nmethods'	Non-heap memory	2.43 MiB	2.43 MiB	5.56 MiB	1.47 MiB (26%)
CodeHeap 'non-profiled nmethods'	Non-heap memory	2.43 MiB	2.43 MiB	117.21 MiB	1.22 MiB (1%)
CodeHeap 'profiled nmethods'	Non-heap memory	2.43 MiB	5.75 MiB	117.21 MiB	5.69 MiB (4%)
Compressed Class Space	Non-heap memory	0.00 MiB	2.00 MiB	1024.00 MiB	1.84 MiB (0%)
Metaspace	Non-heap memory	0.00 MiB	17.87 MiB	-0.00 MiB	17.58 MiB

"http-nio-8080"

Max threads: 200 Current thread count: 10 Current threads busy: 1 Keep alive sockets count: 2
Max processing time: 129 ms Processing time: 0.384 s Request count: 9 Error count: 1 Bytes received: 0.00 MiB Bytes sent: 0.08 MiB

Stage	Time	Bytes Sent	Bytes Recv	Client (Forwarded)	Client (Actual)	VHost	Request
R	?	?	?	?	?	?	
S	3 ms	0 KiB	0 KiB	0:0:0:0:0:0:0:1	0:0:0:0:0:0:0:1	localhost	GET /manager/status HTTP/1.1

Conclusion

This demonstration verified the proper functioning of the Tomcat server by accessing the manager interface with authentication credentials and checking the server status.
Successful login and the display of server metrics confirmed that the server is correctly installed, configured, and running.

LAB 2

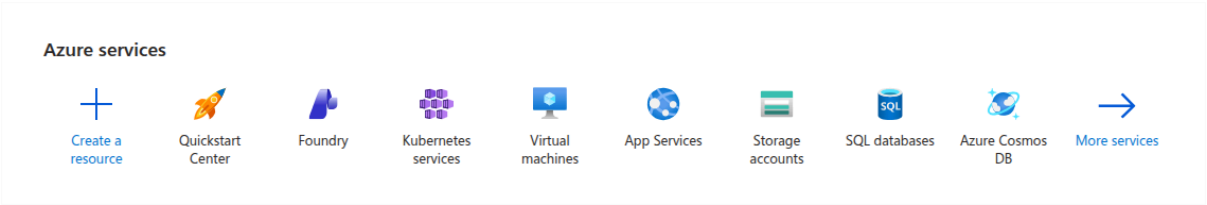
Creating a Virtual Machine on Azure and uploading a file to that Virtual Machine

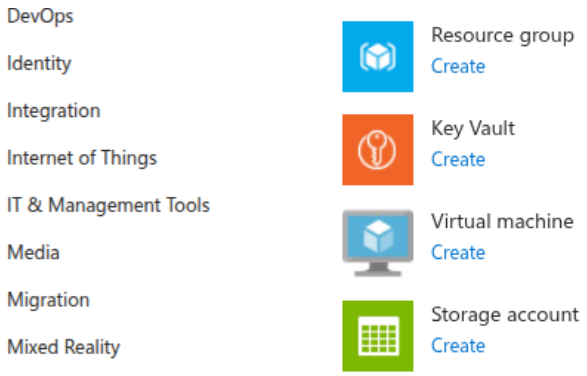
Theory:

An Azure Virtual Machine (VM) is a cloud-based computer provided by Microsoft Azure that runs on Microsoft's servers instead of your local hardware. It allows users to run operating systems, install software, and perform computing tasks just like a physical computer. In simple terms, an Azure VM is a virtual computer in the cloud that you can access and use from anywhere through the internet.

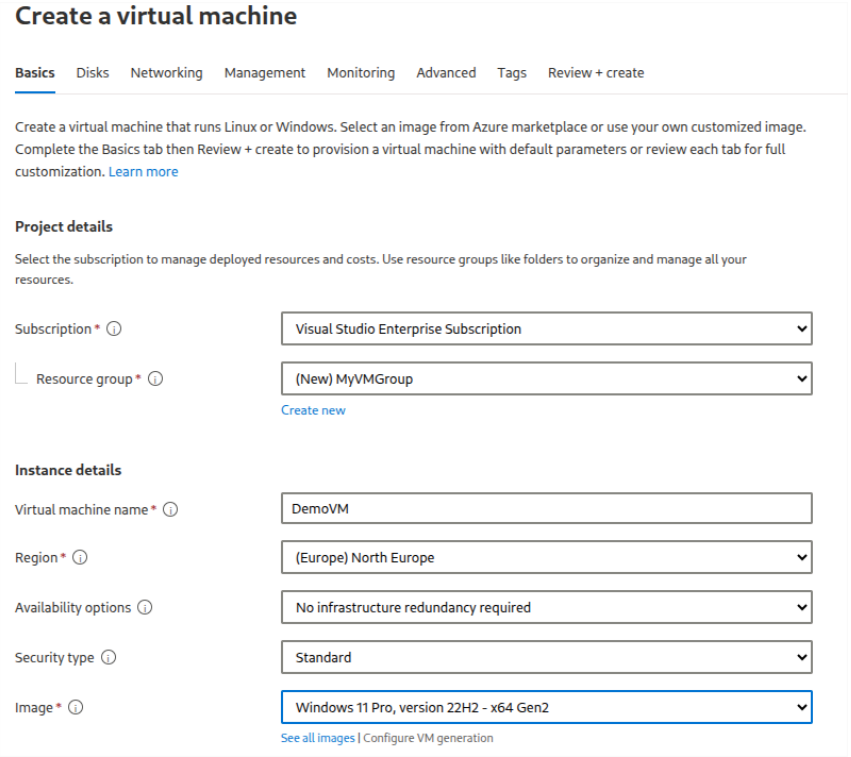
Steps - Creating a Azure VM

1. Go to the Microsoft Azure portal(https://portal.azure.com) and login using Azure Account.
2. To create a vm, click Create a resource and select Virtual Machine.





3. Configure Basic VM Settings. Fill the required details such as Virtual Machine Name, Region,Image, Size of VM, Subscription and so on.



4. Choose authentication type: Username and Password or SSH Public Key. Create the username and password for accessing the VM.
5. Now,configure networking. Allow required ports 22(SSH). Then, click Review and Create to deploy the VM.
6. Connect to the Virtual Machine. Use “ssh username@public-ip-address” to connect to the vm through the terminal.
7. Upload a file to the virtual machine. Use “scp filename username@public-ip-address:/home/username”.
8. Log into the VM and check that the file exists in the destination folder. Use command “ls” to view the files.

Conclusion

This demonstration showed how to create a virtual machine in Azure, configure authentication and networking, and connect to the machine. The process also demonstrated uploading a file to the virtual machine and verifying that the file was successfully transferred.

LAB 3

Hosting a Static Site using Azure Blob Storage

Theory

Microsoft Azure Storage is a cloud service that allows users to store data such as files, images, videos, and documents securely on Microsoft's cloud servers. One feature of Microsoft Azure storage is static website hosting, where users can upload simple website files like HTML, CSS, and JavaScript to Azure Blob Storage. In simple terms, Azure Storage lets you store website files in the cloud and serve them directly on the internet without needing a traditional web server.

Steps - To Host Static Website using Azure Blob Storage

1. Open the Microsoft Azure Portal.
2. Click on Create a Resource. Select Storage Account. Enter details such as:-
    a. Subscription
    b. Resource Group
    c. Storage Account
    d. Name
    e. Region

    Click Review+ Create and then Create.
3. Now, enable static website hosting. Open the created Storage Account. Go to Data Management > Static Website. Enable Static Website Hosting.
4. Now, in the Index Document name, place 'index.html' and in the Error document path, place '404.html' and save the configuration.
5. After enabling static hosting, Azure provides a primary endpoint URL for the website.
6. Open the provided endpoint URL in a browser and verify that the uploaded 'index.html' page loads successfully.

Conclusion

This demonstration showed how to create a storage account, enable static website hosting, and upload web files using Azure Blob Storage. The uploaded files were successfully served as a static website using the Azure storage endpoint.

LAB 4

To get familiar with Azure Services.

Theory

Microsoft Azure provides many cloud services that help developers and organizations build, deploy, and manage applications through Microsoft's cloud platform. These services are grouped into different categories based on their functionality.

Compute Services

These services provide processing power to run applications. Examples include Virtual Machines for running operating systems, App Services for hosting web applications, and Azure Kubernetes Service for containerized applications.

Networking Services

Networking services help connect and manage communication between resources. Azure provides Virtual Network for private networking, Load Balancer for distributing traffic, VPN Gateway for secure connections, and DNS services for domain name resolution.

Storage Services

Storage services allow users to store and manage data in the cloud. Azure offers Blob Storage for unstructured data like images and files, Disk Storage for virtual machines, and File Storage for shared file access across systems.

Database Services

Azure provides managed database solutions for storing structured and unstructured data. Examples include Azure SQL Database for relational data, Cosmos DB for globally distributed databases, and Azure Database for MySQL or PostgreSQL.

AI and Machine Learning Services

These services help developers build intelligent applications. Azure provides Azure Machine Learning for building and training models, Cognitive Services for speech, vision, and language APIs, and AI tools for automation and data analysis.

Other Services

Azure also offers additional services such as Security services for protecting applications and data, DevOps tools for development and deployment automation, and Analytics services for processing large amounts of data.

Conclusion

This demonstration showed how Microsoft Azure offers a comprehensive set of services across compute, networking, storage, databases, AI, and more, enabling users to build and manage cloud applications efficiently. Learning these services helps in understanding how cloud platforms support scalable, secure, and intelligent solutions.

LAB 5

i. Write a program to implement multithreading in Java.

Theory

Multithreading is a feature that allows a program to run multiple threads (smaller units of a process) simultaneously. Each thread can perform a different task at the same time, which makes programs faster and more efficient, especially on multi-core processors. In simple terms, multithreading lets a Java program do many things at once, like downloading a file while also updating the user interface.

Steps

i. Create a new .java extension file and write the code.
ii. Compile the java program using “javac filename.java”.
iii. Execute the code using “java filename”.

Source Code

```
class MyThread extends Thread {
  public void run() {
    for (int i = 1; i <= 5; i++) {
      System.out.println(Thread.currentThread().getName() + " - Count: " + i);
      try {
        Thread.sleep(500);
      } catch (InterruptedException e) {
        e.printStackTrace();
      }
    }
  }
}


public class MultithreadingExample {
  public static void main(String[] args) {
    MyThread t1 = new MyThread();
    MyThread t2 = new MyThread();


    t1.start();
```

```
        t2.start();
    }
}
```

OUTPUT

```
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ javac MultithreadingEx
ample.java
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ java MultithreadingExa
mple

Thread-0 - Count: 1
Thread-1 - Count: 1
Thread-0 - Count: 2
Thread-1 - Count: 2
Thread-0 - Count: 3
Thread-1 - Count: 3
Thread-0 - Count: 4
Thread-1 - Count: 4
Thread-0 - Count: 5
Thread-1 - Count: 5
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$
```

ii. Write a program to implement task programming in .NET.

Theory

Task-based programming is a way of writing code that allows multiple operations to run asynchronously using Task objects. Instead of running tasks one by one, a program can start a task and continue working on other things while the task completes in the background. In simple terms, task programming lets applications perform work efficiently by handling multiple operations at the same time without blocking the main program.

Steps

i. Create a new console project using “dotnet new console -n TaskProgram”.
ii. Open Program.cs and place the source code inside it.
iii. To compile and run the code use “dotnet run”.

Source Code

```
using System;
using System.Threading;
using System.Threading.Tasks;
class Program
{
    static void PrintNumbers()
    {
        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine("Task 1: " + i);
            Thread.Sleep(500);
        }
    }
    static void PrintLetters()
    {
        for (char c = 'A'; c <= 'E'; c++)
        {
            Console.WriteLine("Task 2: " + c);
            Thread.Sleep(500);
        }
    }
    static void Main()
    {
        Task t1 = new Task(PrintNumbers);
        Task t2 = new Task(PrintLetters);

        t1.Start();
```

```
        t2.Start();
        Task.WaitAll(t1, t2);
        Console.WriteLine("Tasks Completed");
    }
}
```

```
using System;
using System.Threading;
using System.Threading.Tasks;

class Program
{
    static void PrintNumbers()
    {
        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine("Task 1: " + i);
            Thread.Sleep(500);
        }
    }

    static void PrintLetters()
    {
        for (char c = 'A'; c <= 'E'; c++)
        {
            Console.WriteLine("Task 2: " + c);
            Thread.Sleep(500);
        }
    }

    static void Main()
    {
        Task t1 = new Task(PrintNumbers);
        Task t2 = new Task(PrintLetters);

        t1.Start();
        t2.Start();

        Task.WaitAll(t1, t2);

        Console.WriteLine("Tasks Completed");
    }
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line
```

Output

```
Welcome to .NET 10.0!
---------------------
SDK Version: 10.0.103

----------------
Installed an ASP.NET Core HTTPS development certificate.
To trust the certificate, run 'dotnet dev-certs https --trust'
Learn about HTTPS: https://aka.ms/dotnet-https

----------------
Write your first app: https://aka.ms/dotnet-hello-world
Find out what's new: https://aka.ms/dotnet-whats-new
Explore documentation: https://aka.ms/dotnet-docs
Report issues and find source on GitHub: https://github.com/dotnet/core
Use 'dotnet --help' to see available commands or visit: https://aka.ms/dotnet-cli
--------------------------------------------------------------------------------------
The template "Console App" was created successfully.

Processing post-creation actions...
Restoring /home/kailashbadu/Desktop/Learning/college/adb/cloud-computing/TaskProgram/TaskP
rogram.csproj:
Restore succeeded.


kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ ls
MultithreadingExample.class  MultithreadingExample.java  MyThread.class  TaskProgram
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ cd TaskProgram
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing/TaskProgram$ nano Program.c
s
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing/TaskProgram$ dotnet run
Task 2: A
Task 1: 1
Task 2: B
Task 1: 2
Task 2: C
Task 1: 3
Task 2: D
Task 1: 4
Task 2: E
Task 1: 5
Tasks Completed
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing/TaskProgram$
```

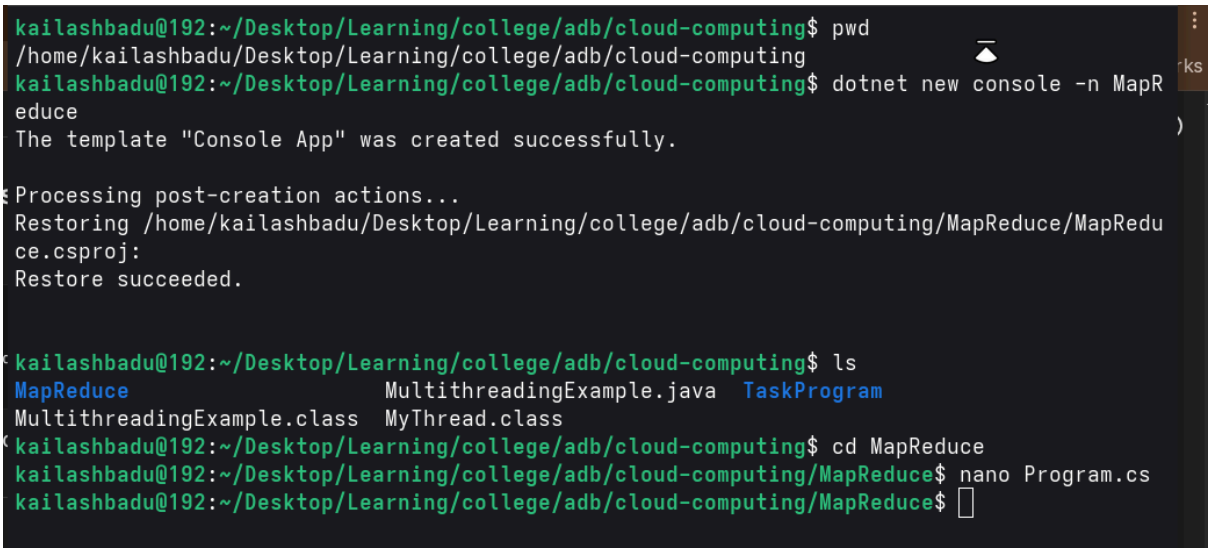
iii. Write a program to implement Hadoop / MapReduce in .NET.

Theory

MapReduce is a programming model used to process large amounts of data by splitting it into smaller chunks, processing them in parallel (Map), and then combining the results (Reduce). It helps handle big data efficiently across multiple machines or cores. In simple terms, MapReduce lets programs analyze and summarize large datasets quickly by breaking tasks into smaller pieces and then merging the results.

Steps

i. Create a new console project using “dotnet new console -n MapReduce”.
ii. Open Program.cs and place the source code inside it.
iii. To compile and run the code use “dotnet run”.



Source Code

```
using System;
using System.Collections.Generic;

class MapReduceExample
{
  // Map Function
  static List<KeyValuePair<string, int>> Map(string text)
  {
    List<KeyValuePair<string, int>> mapped = new List<KeyValuePair<string, int>>();
    string[] words = text.Split(' ');
    foreach (string word in words)
    {
      mapped.Add(new KeyValuePair<string, int>(word.ToLower(), 1));
```

```
        }
        return mapped;
    }

    // Reduce Function
    static Dictionary<string, int> Reduce(List<KeyValuePair<string, int>> mappedData)
    {
        Dictionary<string, int> result = new Dictionary<string, int>();
        foreach (var pair in mappedData)
        {
            if (result.ContainsKey(pair.Key))
                result[pair.Key] += pair.Value;
            else
                result[pair.Key] = pair.Value;
        }
        return result;
    }

    static void Main()
    {
        string text = "hadoop mapreduce hadoop bigdata mapreduce data";
        var mapped = Map(text);
        var reduced = Reduce(mapped);

        Console.WriteLine("Word Count using MapReduce:\n");
        foreach (var item in reduced)
        {
            Console.WriteLine(item.Key + " : " + item.Value);
        }
    }
}
```

OUTPUT

```
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing/MapReduce$ dotnet run
Word Count using MapReduce:

hadoop : 2
mapreduce : 2
bigdata : 1
data : 1
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing/MapReduce$
```

LAB 6

i. Write a program to implement simple Task Programming in Java and execute it in a Virtual Machine (VM).

Theory

Task programming in Java allows a program to run tasks concurrently using threads so multiple operations can execute at the same time. This improves performance and allows efficient use of system resources. The program can be executed inside a Virtual Machine, which provides an isolated environment where the operating system and applications run independently from the host system.

Running the Code on VM:

Steps

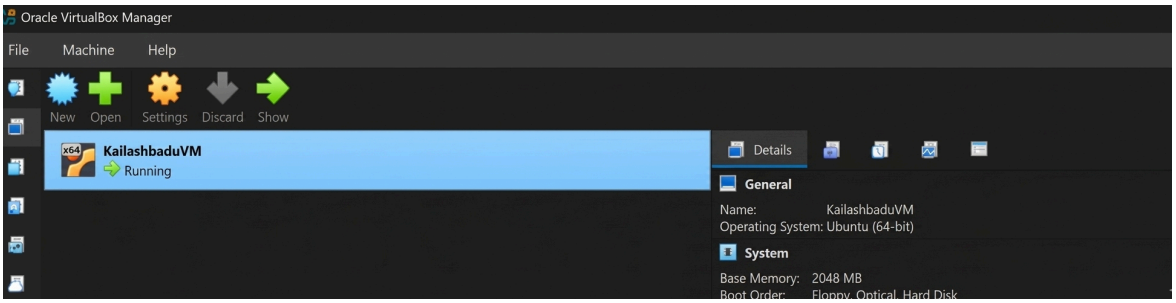
i. Start the Virtual Machine and log in to the operating system installed in it.
ii. Create a new .java extension file and write the code.
iii. Install java using “sudo pacman -S openjdk-8-jdk”.
iv. Set the environment using “export JAVA_HOME=/path/to/jdk” and “export PATH=$PATH:$JAVA_HOME/bin”.
v. Compile the java program using “javac filename.java”.
vi. Execute the code using “java filename”.

Source Code

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
class Task1 implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Task 1 - Number: " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
```

```
class Task2 implements Runnable {
    public void run() {
        for (char c = 'A'; c <= 'E'; c++) {
            System.out.println("Task 2 - Letter: " + c);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class TaskProgramming {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(2);
        executor.execute(new Task1());
        executor.execute(new Task2());
        executor.shutdown();
    }
}
```

OUTPUT:




```
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ pwd
/home/kailashbadu/Desktop/Learning/college/adb/cloud-computing
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ ls
MapReduce                    MultithreadingExample.java  TaskProgram
MultithreadingExample.class  MyThread.class
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ cd TaskProgram/
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing/TaskProgram$ ls
bin  obj  Program.cs  TaskProgram.csproj
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing/TaskProgram$ cd ..
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ touch lab6.java
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ nano lab6.java
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$
```

```
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ javac lab6.java
error: file not found: lab6.java
Usage: javac <options> <source files>
use --help for a list of possible options
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ javac Lab6.java
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ la
bash: la: command not found...
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ ls
Lab6.class  MultithreadingExample.class  Task1.class
Lab6.java   MultithreadingExample.java   Task2.class
MapReduce   MyThread.class               TaskProgram
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ java Lab6.class
Error: Could not find or load main class Lab6.class
Caused by: java.lang.ClassNotFoundException: Lab6.class
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ java Lab6
Task 2 - Letter: A
Task 1 - Number: 1
Task 2 - Letter: B
Task 1 - Number: 2
Task 2 - Letter: C
Task 1 - Number: 3
Task 2 - Letter: D
Task 1 - Number: 4
Task 2 - Letter: E
Task 1 - Number: 5
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$
```

Conclusion

This demonstration showed how a simple task-based Java program can be created, compiled, and executed within a Virtual Machine environment. Running the program in a virtual machine provides an isolated platform for testing and executing concurrent tasks safely.

ii. Run a Simple C Program in Virtual Machine using Type2 Virtualization.

Theory
Type-2 virtualization uses a hypervisor that runs on top of a host operating system. Hypervisor Software such as Oracle VM VirtualBox or VMware Workstation allows a virtual machine to run inside a normal computer so programs can be tested safely in an isolated environment.

Running the Code on VM:

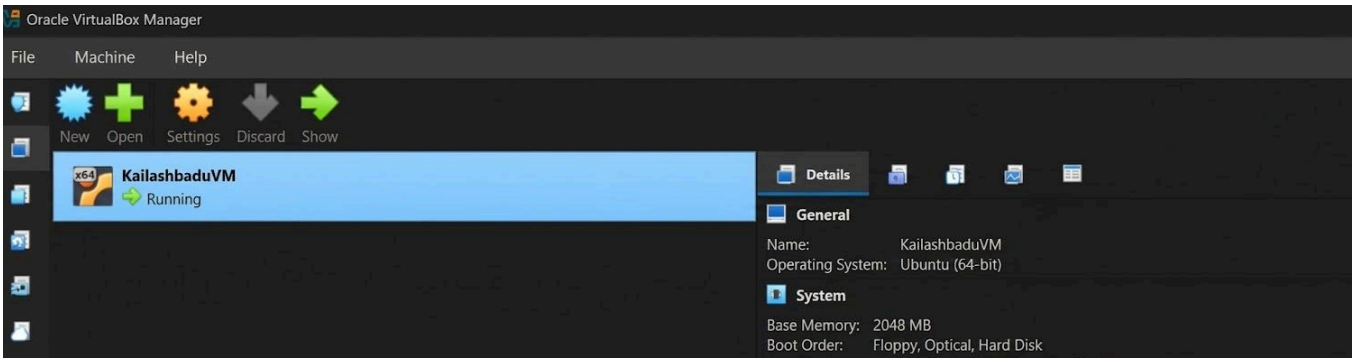
Steps

i.Open the Type-2 virtualization software and start the virtual machine
ii. Create a new .C extension file and write the code.
iii. Install gcc compiler using “sudo pacman -S gcc”..
iv. Compile the c program using “gcc pathtofile -o filename”.
v. Execute the code using “./filename”.

Source Code

```
#include <stdio.h>

int main() {
    int number;

    // Ask the user for input
    printf("Enter an integer: ");
    scanf("%d", &number);

    // Check if number is even or odd
    if (number % 2 == 0) {
        printf("%d is Even.\n", number);
    } else {
        printf("%d is Odd.\n", number);
    }

    return 0;
}
```

OUTPUT:




```
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ pwd
/home/kailashbadu/Desktop/Learning/college/adb/cloud-computing
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ ls
Lab6.class  MultithreadingExample.class  Task1.class
Lab6.java   MultithreadingExample.java   Task2.class
MapReduce   MyThread.class               TaskProgram
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ nano hello.c
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ gcc even_odd.c -o even_odd
cc1: fatal error: even_odd.c: No such file or directory
compilation terminated.
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ gcc hello.c -o hello
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$ ./hello
Enter an integer: 55
55 is Odd.
kailashbadu@192:~/Desktop/Learning/college/adb/cloud-computing$
```

Conclusion

This demonstration showed how to run a simple C program inside a virtual machine using Type-2 virtualization. The program was compiled and executed in the virtual environment, confirming that applications can be developed and tested safely within a virtual machine.

LAB 7

Simulate a cloud scenario using cloud Sim and a scheduling algorithm that is not present in cloud Sim.

Theory

CloudSim is a simulation toolkit used to model and evaluate cloud computing environments. It allows researchers and developers to simulate data centers, virtual machines, and cloud applications without using real cloud infrastructure. In this experiment, a cloud scenario can be simulated using CloudSim by implementing a scheduling algorithm(Priority Scheduling).

CloudSim Simulation Workflow

The simulation generally follows this order:

1. Initialize CloudSim
2. Create Datacenter
3. Create Broker
4. Create Virtual Machines (VMs)
5. Create Cloudlets (Tasks)
6. Broker submits VMs to Datacenter
7. Broker sends Cloudlets to VMs
8. VM executes Cloudlets
9. Results returned to Broker

In CloudSim, the user submits tasks called cloudlets through a broker. The broker communicates with the datacenter to create virtual machines. Once the VM is created, the broker sends cloudlets to the VM for execution. In our experiment, we implemented a custom priority scheduling algorithm so that cloudlets with higher priority are executed before lower priority tasks.

Steps

i. Create a java project named “CloudSimProject”.
ii. Inside the project, create a lib/,bin/ and src/ folder.
iii. Place the “cloudsim.jar” inside the lib/ folder.
iv. Inside the src/ folder we create a custom scheduling algorithm. So, we create a PriorityCloudlet.java,PriorityCloudletSchdeuler.java and PrioritySimulation.java.
v. Compile the simulation using “javac -cp "lib/cloudsim-3.0.3.jar" src/*.java -d bin”.
vi. Now,run the simulation using “java -cp "bin:lib/cloudsim-3.0.3.jar" PrioritySimulation”.

We are defining the cloudlets with their priority as such:-

Cloudlet 0 created with Priority = 5
Cloudlet 1 created with Priority = 2
Cloudlet 2 created with Priority = 9
Cloudlet 3 created with Priority = 1
Cloudlet 4 created with Priority = 7

Source Code

PriorityCloudlet.java

```
import org.cloudbus.cloudsim.Cloudlet;
public class PriorityCloudlet extends Cloudlet {
   private int priority;
   public PriorityCloudlet(int cloudletId, long cloudletLength, int pesNumber,
         long cloudletFileSize, long cloudletOutputSize,int priority) {
      super(cloudletId, cloudletLength, pesNumber,
            cloudletFileSize, cloudletOutputSize,
            null, null, null);
      this.priority = priority;
   }
   public int getPriority() {
      return priority;
   }
}
```

PriorityCloudletSchdeuler.java

```
import org.cloudbus.cloudsim.*;
import java.util.*;
public class PriorityCloudletScheduler extends CloudletSchedulerSpaceShared {
   @Override
   public double cloudletSubmit(Cloudlet cloudlet, double fileTransferTime) {
      ResCloudlet rcl = new ResCloudlet(cloudlet);
      getCloudletExecList().add(rcl);
      // Sort based on priority
      Collections.sort(getCloudletExecList(), new Comparator<ResCloudlet>() {
         public int compare(ResCloudlet r1, ResCloudlet r2) {
            PriorityCloudlet c1 = (PriorityCloudlet) r1.getCloudlet();
            PriorityCloudlet c2 = (PriorityCloudlet) r2.getCloudlet();
            return Integer.compare(c2.getPriority(), c1.getPriority());
```

```
            }
        });
        return super.cloudletSubmit(cloudlet, fileTransferTime);
    }
}
```

PrioritySimulation.java

```
import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.provisioners.*;
import java.util.*;

public class PrioritySimulation {
    public static void main(String[] args) {
        try {
            int numUser = 1;
            Calendar calendar = Calendar.getInstance();
            boolean traceFlag = false;
            CloudSim.init(numUser, calendar, traceFlag);
            Datacenter datacenter = createDatacenter("Datacenter_0");
            DatacenterBroker broker = new DatacenterBroker("Broker");
            int brokerId = broker.getId();
            List<Vm> vmList = new ArrayList<>();
            int vmid = 0;int mips = 1000;long size = 10000;int ram = 512;
            long bw = 1000;int pesNumber = 1;
            Vm vm = new Vm(vmid, brokerId, mips, pesNumber, ram, bw, size,
                    "Xen", new PriorityCloudletScheduler());
            vmList.add(vm);
            broker.submitVmList(vmList);
            List<Cloudlet> cloudletList = new ArrayList<>();
            for(int i=0;i<5;i++) {
                PriorityCloudlet cloudlet = new PriorityCloudlet(
                        i,4000,1,300,300,new Random().nextInt(10));
                cloudlet.setUserId(brokerId);
                cloudletList.add(cloudlet);
            }
            broker.submitCloudletList(cloudletList);
            CloudSim.startSimulation();
            List<Cloudlet> newList = broker.getCloudletReceivedList();
            CloudSim.stopSimulation();
            printCloudletList(newList);
```

```
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private static Datacenter createDatacenter(String name) {
        List<Host> hostList = new ArrayList<>();
        List<Pe> peList = new ArrayList<>();
        peList.add(new Pe(0, new PeProvisionerSimple(1000)));
        int hostId = 0;int ram = 2048;
        long storage = 1000000;
        int bw = 10000;
        hostList.add(new Host(hostId,new RamProvisionerSimple(ram),
                new BwProvisionerSimple(bw),storage,peList,
                new VmSchedulerTimeShared(peList)));
        String arch = "x86";
        String os = "Linux";
        String vmm = "Xen";
        DatacenterCharacteristics characteristics = new DatacenterCharacteristics(
                arch, os, vmm, hostList, 10.0, 3.0, 0.05, 0.1, 0.1);
        try {
            return new Datacenter(name, characteristics,
                    new VmAllocationPolicySimple(hostList),
                    new LinkedList<>(), 0);
        } catch (Exception e) {
            e.printStackTrace();
        }
        return null;
    }

    private static void printCloudletList(List<Cloudlet> list) {
        System.out.println("Cloudlet Results");
        for(Cloudlet cloudlet : list) {
            System.out.println(
                    "Cloudlet ID: " + cloudlet.getCloudletId() +
                    " Status: " + cloudlet.getStatus() +
                    " Finish Time: " + cloudlet.getFinishTime()
            );
        }
    }
}
```

OUTPUT

Defining the Broker and the Cloudlets

```
PS D:\Cloudlab> java -cp "bin;lib/cloudsim-3.0.3.jar" PrioritySimulation
Initialising...
Starting CloudSim version 3.0
Datacenter_0 is starting...
Broker is starting...
Entities started.
```

VM Creation by Datacenter and Broker Sending Cloudlets to VM

```
0.0: Broker: Cloud Resource List received with 1 resource(s)
0.0: Broker: Trying to Create VM #0 in Datacenter_0
0.1: Broker: VM #0 has been created in Datacenter #2, Host #0
0.1: Broker: Sending cloudlet 0 to VM #0
0.1: Broker: Sending cloudlet 1 to VM #0
0.1: Broker: Sending cloudlet 2 to VM #0
0.1: Broker: Sending cloudlet 3 to VM #0
0.1: Broker: Sending cloudlet 4 to VM #0
4.1: Broker: Cloudlet 4 received
4.1: Broker: Cloudlet 2 received
4.1: Broker: Cloudlet 1 received
4.1: Broker: Cloudlet 0 received
4.1: Broker: Cloudlet 0 received
4.1: Broker: All Cloudlets executed. Finishing...
4.1: Broker: Destroying VM #0
4.1: Broker: Cloudlet 3 received
Broker is shutting down...
```

Cloudlet Results

```
Cloudlet Results
Cloudlet ID: 4 Status: 3 Finish Time: 4.1
Cloudlet ID: 2 Status: 3 Finish Time: 4.1
Cloudlet ID: 1 Status: 3 Finish Time: 4.1
Cloudlet ID: 0 Status: 4 Finish Time: 4.1
Cloudlet ID: 0 Status: 4 Finish Time: 4.1
Cloudlet ID: 3 Status: 3 Finish Time: 4.1
PS D:\Cloudlab>
```

LAB 8

Create a SOAP based web service “calculator service” in java and demonstrate the result.

Theory

SOAP (Simple Object Access Protocol) is a messaging protocol used to allow different applications to communicate with each other over the internet. It uses XML format to send requests and responses between a client and a server, usually through HTTP. In simple terms, SOAP acts like a structured message system that lets software systems exchange information reliably across different platforms and programming languages.

Steps

i. Create the server side java code. One for services and one for publisher.
ii. Compile the SOAP service program by “javac -d . CalculatorService.java CalculatorPublisher.java”
iii. Run the SOAP web service by “java calculator.CalculatorPublisher“.
iv. Generate client files using WSDL(“wsimport -keep http://localhost:8080/calculator?wsdl”). This generates the client stub classes.
v. Create the client side java program.
vi. Compile the client program by “javac CalculatorClient.java” and execute the client program by “java CalculatorClient”.

Source Code

CalculatorService.java

```
package calculator;

import javax.jws.WebMethod;
import javax.jws.WebService;

@WebService
public class CalculatorService {

    @WebMethod
    public int add(int a, int b) {
        return a + b;
    }

    @WebMethod
    public int subtract(int a, int b) {
        return a - b;
    }
```

```
    @WebMethod
    public int multiply(int a, int b) {
        return a * b;
    }

    @WebMethod
    public int divide(int a, int b) {
        return a / b;
    }
}
```

CalculatorPublisher.java

```
package calculator;

import javax.xml.ws.Endpoint;

public class CalculatorPublisher {

    public static void main(String[] args) {

        Endpoint.publish(
            "http://localhost:8080/calculator",
            new CalculatorService()
        );

        System.out.println("Calculator SOAP Service Running...");
    }
}
```

CalculatorClient.java

```
import calculator.CalculatorService;
import calculator.CalculatorServiceService;

public class CalculatorClient {

    public static void main(String[] args) {

        CalculatorServiceService service = new CalculatorServiceService();
        CalculatorService calculator = service.getCalculatorServicePort();

        int a = 10;
        int b = 5;

        System.out.println("Addition: " + calculator.add(a, b));
        System.out.println("Subtraction: " + calculator.subtract(a, b));
```

```
        System.out.println("Multiplication: " + calculator.multiply(a, b));
        System.out.println("Division: " + calculator.divide(a, b));
    }
}
```

OUTPUT:

SOAP Server Running

```
PS C:\Users\ACER\Desktop\javas\CalculatorSOAP> java calculator.CalculatorPublisher
Calculator SOAP Service Running...
```

SOAP Client Result

```
PS C:\Users\ACER\Desktop\javas\CalculatorSOAP> java CalculatorClient
Addition: 26
Subtraction: -18
Multiplication: 88
Division: 0
```